BhashaWrite: A Transparent Multilingual Writing Assistant for Roman-Script Indian Languages

CS3235 — Working with Large Language Models | Mini Project 4 | CIE3 Lab

Student name: [INSERT YOUR NAME]
USN / Roll number: [INSERT YOUR USN]
Department: B.Tech (H), Computer Science and Engineering
Submission date: 6 November 2026
Course: CS3235 — Working with Large Language Models

> **Report type:** Undergraduate software-engineering mini-project report. Numerical results refer to a fixed, tiny offline-demo baseline and are not claims of learned-model performance. Large AI4Bharat/BharatGen pretrained models have not been downloaded or benchmarked here.

Abstract

Roman-script typing is common in daily Indian-language digital communication. Its flexibility poses challenges for an editor: a spoken Hindi word may have multiple Romanized spellings; a sentence may contain English technical names and time expressions; and direct script conversion does not guarantee natural phrasing. This project, **BhashaWrite**, prototypes a modular writing assistant for Hindi, Kannada and Tamil. A Streamlit web frontend accepts Roman input and produces native script. The system supports a transparently labeled offline demonstration based on a small curated lexicon and implements adapters for AI4Bharat IndicXlit transliteration, AI4Bharat IndicTrans2 translation, and BharatGen Param-1-2.9B-Instruct for optional Hindi editing. A common tokenization and protection layer preserves URLs, email addresses, numeric strings and common technical terms. To make evaluation reproducible, the repository contains 45 illustrative sentence cases (15 per language), with character error rate (CER), position-aligned word accuracy and exact sentence match metrics. Locally run offline-demo evaluation achieved mean CER **0.0405**, word accuracy **97.04%**, and exact match **91.11%** on this deliberately small authored set; four cases illustrate loanword/code-mix failures. A 20-test pytest suite passed. Important limitations include small baseline vocabulary, limited external validity, Roman spelling ambiguity and untested neural inference adapters. The main contribution is an inspectable end-to-end educational prototype with explicit model provenance, structured testing and reproducible error analysis.

Keywords: Romanization, transliteration, IndicXlit, AI4Bharat, IndicTrans2, BharatGen, Param1, Streamlit, Character Error Rate, code-switching.

1. Introduction and motivation

Mobile-first communication routinely mixes Roman-script Hindi, Kannada and Tamil with English technical words. Students may type `mujhe paani chahiye`, `nanna hesaru` or `enakku thanni venum`; native-script publishing, captions and accessible text require a reliable transformation. A naive character substitution approach misses word-level pronunciation, context and script conventions. An LLM-based rewriting step may improve grammar but can also change meaning or invent information,

so it must be separable and auditable. The stated CS3235 project asks for native-script conversion, improved phrasing and optional English translation, along with meaningful tests and a usable frontend.

1.1 Problem statement

Create a locally runnable, auditable writing assistant that (1) accepts Romanized Hindi, Kannada or Tamil, (2) produces corresponding native script, (3) performs safe punctuation cleanup, (4) optionally improves Hindi phrasing using a verified generative model and (5) optionally translates to English using a translation model. Report limitations rather than making unsupported accuracy claims.

1.2 Project objectives

1. Provide a usable web interface for three supported languages with downloadable output.
2. Integrate real AI4Bharat/BharatGen checkpoints through modular model adapters, while keeping setup reproducible.
3. Preserve URLs, emails, numbers and selected technical vocabulary.
4. Collect 30–50 representative evaluation cases and score outputs with well-defined metrics.
5. Implement functional tests, failure analysis, user warnings and documentation.

1.3 Scope and non-goals

The application is a student prototype, not a commercial translation service. It does not train a new model, run an external cloud deployment, guarantee grammatical correctness, or substitute for human editing on consequential documents. The built-in offline lexicon is a fallback/demo only and is not presented as AI4Bharat inference.

2. Background and related work

AI4Bharat IndicXlit is a neural transliteration system providing Roman-to-Indic and Indic-to-Roman conversion APIs. Its Python wrapper exposes `XlitEngine.translit_word` and sentence helpers. The official Aksharantar corpus is a better choice for future benchmark evaluation. **AI4Bharat IndicTrans2** provides Indic-English and other translation checkpoints. This project chooses the distilled Indic-to-English 200M checkpoint for a more manageable optional task. **BharatGen Param-1-2.9B-Instruct** is an instruction-following Hindi/English text model; it can be served through an OpenAI-compatible vLLM interface for optional correction. Importantly, the chosen Param1-Instruct checkpoint should not be advertised as a Kannada/Tamil editor. The project separates deterministic text processing from pretrained inference so failures can be assigned to specific stages.

3. Requirements and system design

3.1 Functional requirements

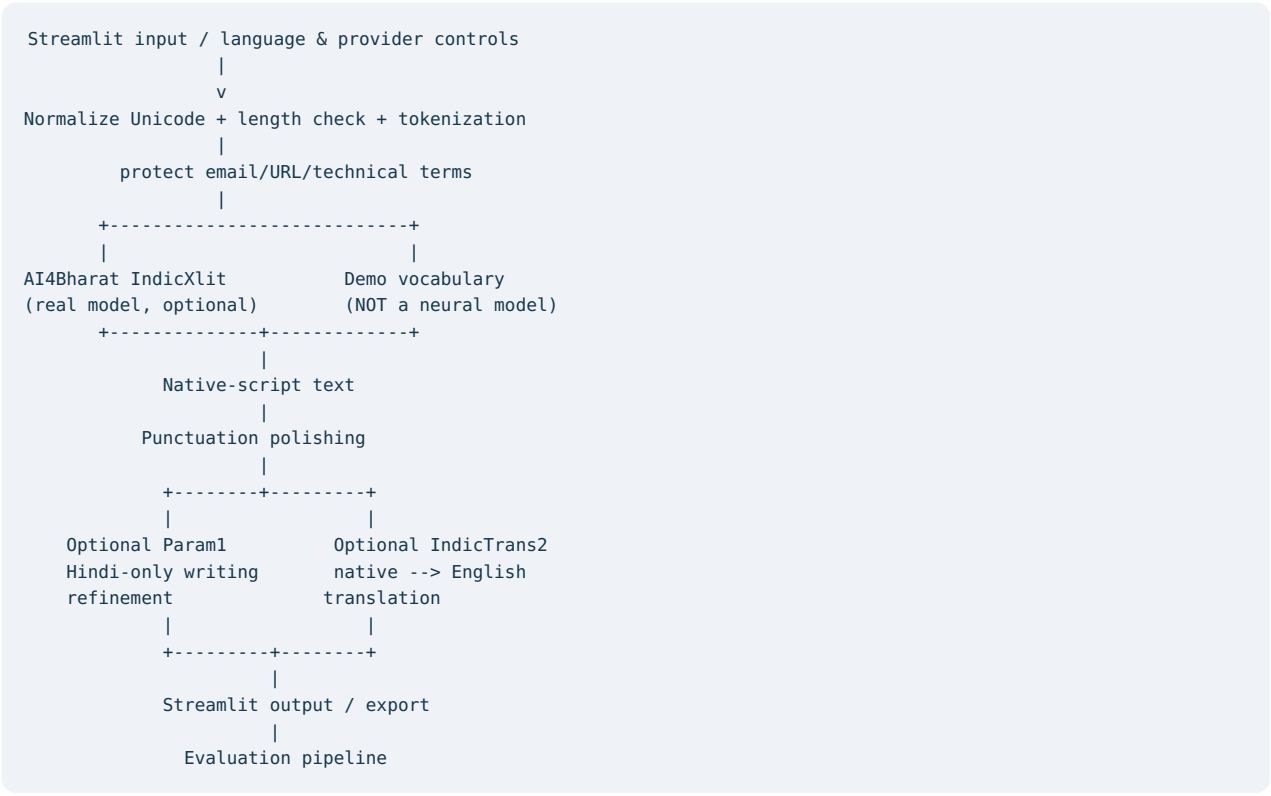
- FR1: Accept up to 1,800 characters of Roman-script input.
- FR2: Support Hindi, Kannada and Tamil script selection.
- FR3: Select actual IndicXlit inference or explicitly labeled offline demonstration.
- FR4: Protect common English technical terms, addresses/URLs, numbers and times.
- FR5: Produce native text and whitespace/punctuation-polished text.
- FR6: Optionally ask BharatGen Param1 to edit Hindi while preserving meaning.
- FR7: Optionally translate native-language text to English via IndicTrans2.
- FR8: Display provider status and explicit errors or warnings.

- FR9: Export results and evaluation tables.

3.2 Nonfunctional requirements

The solution should run without model weights in demo mode, have bounded input, be testable without a GPU, not silently fall back when a neural model is selected, and avoid saving user content to a server database. For hosted public deployment, authentication and rate limiting must be added.

3.3 Architecture



3.4 Component responsibilities

`core/pipeline.py` handles validation, tokenization, source protection and orchestration. `core/providers.py` contains the real model boundaries and errors. `core/lexicon.py` contains the hand-curated offline demonstration vocabulary. `core/metrics.py` implements evaluation independently of model availability. `evaluation/run_eval.py` scores every reference row in a reproducible order. The web frontend is `app.py` and tests live in `tests/`.

4. Implementation

4.1 Text pre-processing and token protection

The input is Unicode NFC normalized and bounded to 1,800 characters. A regular-expression tokenizer identifies Roman words, email addresses, URLs and numerals while preserving punctuation as nonword tokens. URLs/email/numbers pass through unchanged. Common technical terms including GitHub, Python, LLM and API may be protected with a user-controlled switch. Nonprotected Roman words are dispatched to either IndicXlit or the offline demo lexicon.

4.2 Model adapters

Real IndicXlit uses `XlitEngine(lang, beam_width=5, rescore=True)` followed by `translit_word(word, topk=1)`. This requires optional model assets and compatible older Fairseq-related dependencies. Real IndicTrans2 uses `IndicProcessor` preprocessing and postprocessing plus a Hugging Face

encoder-decoder checkpoint. Real Param1 uses a separately deployed vLLM OpenAI-compatible Chat Completions endpoint. The HTTP call is guarded by a configured endpoint, a low-temperature editing prompt and a timeout. The app never manufactures content if a requested model is not available.

4.3 The offline demo baseline

The 3-language lexicon is small enough to audit line by line. Recognized words are replaced with native-script entries; unseen Roman words are retained unchanged and added to a warning list. This produces a real end-to-end runnable demonstration without GPU/network access. However, it is intentionally **not** comparable to a pretrained model, and the small curated evaluation set overestimates general-use accuracy.

4.4 Frontend

Streamlit provides a language selector, provider selector, English-token-preservation toggle, optional refinement and translation toggles, input text area, output sections, warning messages, download control, and evaluation tab. A third information tab explains architecture and limitations. The UI is intentionally explicit about which provider generated the text.

5. Experimental methodology

5.1 Test data

`evaluation/cases.csv` contains **45 authored reference sentences**, evenly divided among Hindi, Kannada and Tamil. Scenarios include greeting, self-introduction, education, request/command, question, punctuation, code-mixed text, numeric time and unseen loanwords. They are practical educational test cases, not an official Aksharantar benchmark, and were not independently annotated by native speakers. Results should be treated as a *smoke-test baseline*, not a generalization estimate.

5.2 Metrics

Let R be the reference string and P be the prediction. **CER = edit_distance(P,R) / max(1, length(R))**, after Unicode NFC normalization. Edit distance counts insertions, deletions and substitutions. **Word accuracy** computes equality of position-aligned whitespace tokens, divided by the maximum token count. **Exact match** is binary equality after NFC normalization and boundary trimming. Word accuracy here is explicitly not standard WER. These metrics can penalize semantically acceptable variants and should be augmented with human ratings.

5.3 Evaluation procedure

```
python -m pytest -q
python evaluation/run_eval.py --mode demo --output evaluation/results-demo.csv
# After separately installing actual IndicXlit:
python evaluation/run_eval.py --mode indicxlit --output evaluation/results-indicxlit.csv
```

All files and source outputs are included. The pretrained-model command was NOT executed in the provided sandbox due to missing checkpoint dependencies/assets; consequently no IndicXlit CER is claimed.

6. Results

Verified local result: offline demo baseline (45 cases).

Language	Cases	Mean CER (lower)	Word accuracy (higher)	Exact match
Hindi	15	0.0571	95.56%	86.67%
Kannada	15	0.0351	97.78%	93.33%
Tamil	15	0.0293	97.78%	93.33%
Overall	**45**	**0.0405**	**97.04%**	**91.11%**

The **20 automated tests passed**, covering three-language conversion, punctuation, protected text, OOV warnings, empty/invalid inputs, 1,800-character boundary enforcement, translation failure behavior, unsupported Param1 language, dataset coverage and scoring. These tests demonstrate working local code paths, not full neural inference reliability.

6.1 Observed failure rows

- **H13:** `certificate` remains Roman in offline mode; reference expects a Hindi loanword transliteration.
- **H14:** `Email` is preserved by design as an English technical term; reference requires Hindi script.
- **K13:** `assignment` remains Roman; the Kannada target uses a native-script loanword.
- **T13:** `assignment` remains Roman; the Tamil target uses a native-script loanword.

These four failures make the limitations of fixed word dictionaries and token preservation visible. The high exact-match figure primarily reflects test data built around known vocabulary, not broad accuracy on new user content.

7. Discussion and failure analysis

7.1 Roman spelling ambiguity

A sound may appear as multiple Roman spellings (for example `namaste` vs `namasthe`), and isolated words lack sentence context. Top-k suggestions and user choice would address ambiguity more reliably than forcing one result.

7.2 Code-mixing and proper names

The system preserves English technical terms to prevent damage to URLs, product names and code. This can conflict with a user expecting all native script. A user-selectable preserve toggle partially addresses the trade-off; name dictionaries and manual editing are recommended.

7.3 Generative editing risks

A Hindi editor can rewrite wording and inadvertently change named entities, numbers, deadlines or meaning. The Param1 prompt instructs minimal editing and treats input as untrusted data, but this is not a formal guarantee. The UI should visibly separate raw transliteration and polished text, and consequential text should be human-reviewed.

7.4 Validity threats

The dataset is tiny, nonrandom, and largely matched to a curated vocabulary. It excludes many dialects, uncommon inflections, long documents and noisy keyboard input. Offline baseline performance is therefore not an estimate of production performance. The model adapters have not

been exercised on actual downloaded weights in this environment. For a stronger study, score an independent Aksharantar test split, collect multiple native-speaker judgments, benchmark neural latency, and preserve full logs/version identifiers.

8. Security, ethics and sustainability

Input is constrained; demo processing is local; no user text is stored in an app database. External models may use substantial disk, RAM/GPU and energy, so the project treats them as opt-in. If running a public service, add authentication, endpoint rate limits, request budgets, logging with personal data redaction, and documented data-retention policies. Check licenses separately for all checkpoints and datasets. Avoid forwarding personal data to untrusted inference endpoints. Translation and rewriting require human oversight for educational or governmental communications.

9. Deployment and reproducibility

The lightweight frontend runs on Python with `streamlit`, `pandas`, `requests` and `pytest`. Model dependencies are split into optional requirements files, allowing a quick offline demo and separate GPU model deployments. GitHub Actions runs unit tests and a demo evaluation script on pushes/pull requests. Publishing the repository and deploying the Streamlit app to a public URL remain user actions; neither is represented as already completed.

10. Conclusion and future work

BhashaWrite implements a student-scale, end-to-end, inspectable writing interface across Hindi, Kannada and Tamil, with a real-model integration design and a transparent offline baseline. The project contributes a usable UI, clear provider distinctions, reproducible 45-case evaluation, 20 verified tests and specific observed failure categories. Immediate next steps are actual IndicXlit inference validation, a held-out benchmark such as Aksharantar, interactive top-k alternatives, human review of quality, tighter code-mixing controls, and measured latency/cost reports for BharatGen Param1 and IndicTrans2.

References

[1] AI4Bharat, *IndicXlit*. https://github.com/AI4Bharat/IndicXlit
[2] AI4Bharat, *Aksharantar dataset*. https://huggingface.co/datasets/ai4bharat/Aksharantar
[3] AI4Bharat, *IndicTrans2*. https://github.com/AI4Bharat/IndicTrans2
[4] Varun Gumma et al., *IndicTransToolkit*. https://github.com/VarunGumma/IndicTransToolkit
[5] BharatGen, *Param-1-2.9B-Instruct model card*. https://huggingface.co/bharatgenai/Param-1-2.9B-Instruct
[6] BharatGen, *Models and deployment requirements*. https://bharatgen.com/models/
[7] CS3235, *Working with Large Language Models — Lesson Plan and CIE Rubric*, supplied course material, 2026.

Appendix A: Files included

`app.py`, `core/`, `evaluation/`, `tests/`, `.github/workflows/tests.yml`, `requirements*.txt`, `docs/`, `.streamlit/config.toml`, `README.md`, `.gitignore`, `.env.example`, `LICENSE`.

Appendix B: Individualization and authenticity

Replace title placeholders with your actual details; publish your own repository; run the UI and capture an authentic demo screenshot; rerun evaluation when using actual IndicXlit. The report deliberately discloses model inference that has not been executed rather than fabricating outcomes.